

Chapter 4 Exercises

Legrand Jones II

1

a

We can define the predecessor function on $\mathbb{Z}$, recalling remark 4.5.2 (i.e. using our iterated inductions principles from the 3 inductive type constructions involved in our definition of $\mathbb{Z}$), from the following definitions:

$$\text{pred}(-1_{\mathbb{Z}}) := -2_{\mathbb{Z}}$$

$$\text{pred}(\text{in-neg}(\text{succ}_{\mathbb{N}}(n))) := \text{in-neg}(\text{succ}_{\mathbb{N}}(\text{succ}_{\mathbb{N}}(n)))$$

$$\text{pred}(0_{\mathbb{Z}}) := -1_{\mathbb{Z}}$$

$$\text{pred}(1_{\mathbb{Z}}) := 0_{\mathbb{Z}}$$

$$\text{pred}(\text{in-pos}(\text{succ}_{\mathbb{N}}(n))) := \text{in-pos}(n)$$

b

We define $\text{add}_{\mathbb{Z}}$ and $\text{neg}_{\mathbb{Z}}$ in a similar manner to what was done for the previous part, though fix some $m : \mathbb{Z}$, defining $\text{add}_{\mathbb{Z}}(m) : \mathbb{Z} \rightarrow \mathbb{Z}$:

$$\text{add}_{\mathbb{Z}}(m)(\text{in-neg}(\text{succ}_{\mathbb{N}}(n))) := \text{pred}(\text{add}_{\mathbb{Z}}(m)(\text{in-neg}(n)))$$

$$\text{add}_{\mathbb{Z}}(m)(-1_{\mathbb{Z}}) := \text{pred}(m)$$

$$\text{add}_{\mathbb{Z}}(m)(0_{\mathbb{Z}}) := m$$

$$\text{add}_{\mathbb{Z}}(m)(1_{\mathbb{Z}}) := \text{pred}(m)$$

$$\text{add}_{\mathbb{Z}}(m)(\text{in-pos}(\text{succ}_{\mathbb{N}}(n))) := \text{succ}(\text{add}_{\mathbb{Z}}(m)(n))$$

$$\text{neg}_{\mathbb{Z}}(\text{in-neg}(\text{succ}_{\mathbb{N}}(n))) := \text{in-pos}(\text{succ}_{\mathbb{N}}(n))$$

$$\text{neg}_{\mathbb{Z}}(-1_{\mathbb{Z}}) := 1_{\mathbb{Z}}$$

$$\text{neg}_{\mathbb{Z}}(0_{\mathbb{Z}}) := 0_{\mathbb{Z}}$$

$$\text{neg}_{\mathbb{Z}}(1_{\mathbb{Z}}) := -1_{\mathbb{Z}}$$

$$\text{neg}_{\mathbb{Z}}(\text{in-pos}(\text{succ}_{\mathbb{N}}(n))) := \text{in-neg}(\text{succ}_{\mathbb{N}}(n))$$

We finish specifying $\text{add}_{\mathbb{Z}}$ by using the same idea from remark 4.5.2, describing it recursively, by just replacing m in the above with the proper generators.

c

Multiplication on our current model of $\mathbb{Z}$ can be defined inductively in much the same way as above with addition. The main difference is that in our last step of the previous part, for addition, we now have to actually break into different cases for different constructors (for example we need to reflect that multiplication by 0 is 0).

2

a

Via weakening we can index bool over itself. By our induction principle for bool we then have

$$i := \text{bool} \rightarrow (\text{bool} \rightarrow \Pi_{(x:\text{bool})}\text{bool})$$

together with some computation rules. In particular our computation rules say that $i(\text{true}, \text{false})(\text{false}) = \text{true}$ and $i(\text{true}, \text{false})(\text{true}) = \text{false}$. We thus define $\text{neg} - \text{bool} := i(\text{true}, \text{false}) : \Pi_{(x:\text{bool})}\text{bool}$.

b

We use i as above but define $\text{conj} - \text{bool} := i(\text{false}, _) : \text{bool} \rightarrow \Pi_{(x:\text{bool})}\text{bool}$ (our notation is a bit bad here). We note that this behaves as expected because $i(\text{false}, \text{true})(\text{true}) = \text{true}$ while

$$i(\text{false}, \text{false})(\text{true}) = \text{false}$$

$$i(\text{false}, \text{true})(\text{false}) = \text{false}$$

$$i(false, false)(false) = false$$

all coming from our computation rules.

c

We can compose our two previous constructions to get our disjunction function.

3

a

(i)

$$\frac{\frac{z : (P \times \neg P) \vdash pr_1(z) := x_1 : P \quad z : (P \times \neg P) \vdash pr_2(z) := x_2 : \neg P}{z : (P \times \neg P) \vdash x_2(x_1) : \emptyset}}{\vdash \lambda z. x_2(x_1) : \prod_{(z : (P \times \neg P))} \emptyset} \lambda$$

Where we recall that definitionally $\prod_{z : P \times \neg P} \emptyset =: (P \times \neg P) \rightarrow \emptyset =: \neg(P \times \neg P)$. Note also that in evaluating *at* some term which follows from our context we are using a combination of substitution and evaluation (the latter just being for product types). I.e. we need to combine these to say something like the following:

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash f : A \rightarrow B}{\Gamma \vdash f(a) : B} eval$$

(We can add a context before everything for the situation where P is a type)

(ii)

(At the start of the derivation we omit some quick pieces about having our projection and assuming that in our context we have a term of type $P \leftrightarrow \neg P$. As before we also omit the context Γ for which we presume all of our types and such are derivable. We also omit some situations where we apply the last mentioned rule in the above.)

$$\frac{\frac{z : (P \leftrightarrow \neg P) \vdash x_2 := pr_2(z) : \neg P \rightarrow P}{z : (P \leftrightarrow \neg P), q : \neg P \vdash x_2(q) : P} ev}{z : (P \leftrightarrow \neg P) \vdash \lambda q. q(x_2(q)) : (\neg P \rightarrow \emptyset)} eval + \lambda \quad \frac{\frac{z : (P \leftrightarrow \neg P) \vdash x_1 := pr_1(z) : P \rightarrow \neg P}{z : (P \leftrightarrow \neg P), p : P \vdash x_1(p) : \neg P} ev}{z : (P \leftrightarrow \neg P) \vdash \lambda p. x_1(p)(p) : \neg P} eval + \lambda}{\vdash \lambda z. (\lambda q. q(x_2(q)))(\lambda p. x_1(p)(p)) : (P \leftrightarrow \neg P) \rightarrow \emptyset} eval + \lambda$$

We use *ev* and *eval* to distinguish between the ideas of *ev* from the reference text and *eval* from the derivation at the end of the previous part.

b

(i)

(At the start of the derivation we omit some fiddling with introducing variables and applying the eval type derivation from a(i). We also again omit the context that our types derive from.)

$$\frac{p : P, q : \neg P \vdash q(p) : \emptyset}{\vdash \lambda p. \lambda q. q(p) : P \rightarrow \neg \neg P} \lambda^2$$

(ii)

$$\frac{f : P \rightarrow Q, p : \neg \neg P, q : \neg Q \vdash p(\text{comp}(f, q)) : \emptyset}{\lambda f. \lambda p. \lambda q. p(\text{comp}(f, q)) : (P \rightarrow Q) \rightarrow (\neg \neg P \rightarrow \neg \neg Q)} \lambda^3$$

(iii)

This follows from the previous two parts after we establish $\neg \neg \neg Q \rightarrow \neg Q$, using this to compose at the end. This can also be derived as follows:

$$\frac{f : P \rightarrow \neg \neg Q, p : \neg \neg P, q : \neg Q \vdash p : \neg \neg P \quad \frac{q : \neg Q, f : (P \rightarrow \neg \neg Q), p : \neg \neg P, s : P \vdash f(s)(q) : \emptyset}{q : \neg Q, f : (P \rightarrow \neg \neg Q), p : \neg \neg P \vdash \lambda s. f(s)(q) : \neg P} \lambda}{\frac{f : P \rightarrow \neg \neg Q, p : \neg \neg P, q : \neg Q \vdash p(\lambda s. f(s)(q)) : \emptyset}{\vdash \lambda f. \lambda p. \lambda q. p(\lambda s. f(s)(q)) : (P \rightarrow \neg \neg Q) \rightarrow (\neg \neg P \rightarrow \neg \neg Q)} \lambda^3} eval$$

(We are a bit loose with ordering premises in the above when they do not actually depend on each other/can be interchanged)

c

(i)

(To make the following slightly more compact we omit the types of some terms after they are first specified)

$$\frac{s, p : P \vdash s \quad \frac{s : \neg(\neg \neg P \rightarrow P), p : P, r : \neg \neg P \vdash p : P}{s, p : P \vdash \lambda r. p : (\neg \neg P \rightarrow P)} \lambda}{s, p \vdash s(\lambda r. p) : \emptyset} eval \quad \frac{s, p \vdash s(\lambda r. p) : \emptyset}{s \vdash \lambda p. (s(\lambda r. p)) : \neg P} \lambda$$

We thus have a term of type $\neg P$ in context $s : \neg(\neg \neg P \rightarrow P)$. Let us denote our term by $f : \neg P$. We can then continue

$$\frac{s, q \vdash \text{ex-falso-P} : \emptyset \rightarrow P \quad \frac{s, q : \neg \neg P \vdash \neg \neg P \quad s, q \vdash f : \neg P}{s, q \vdash q(f) : \emptyset} eval}{s, q \vdash \text{ex-falso-P}(q(f)) : P} eval \quad \frac{s, q \vdash \text{ex-falso-P}(q(f)) : P}{s \vdash \lambda q. \text{ex-falso-P}(q(f)) : \neg \neg P \rightarrow P} \lambda$$

and finally reach our desired conclusion/construction by using the term we constructed of type $\neg\neg P \rightarrow P$ in context s by evaluation of s at the term, to get a term in $\emptyset$, then form a λ -abstraction to get our final term of our desired type in the empty context (or whatever context P types).

(ii)

(To make the following slightly more compact we omit the types of some terms after they are first specified)

$$\frac{s, p \vdash \text{ex-falso-Q} : \emptyset \rightarrow Q \quad \frac{s, p \vdash s \quad \frac{s : \neg(((P \rightarrow Q) \rightarrow P) \rightarrow P), r : ((P \rightarrow Q) \rightarrow P), p : P \vdash p : P}{s, p : P \vdash \lambda r.p : (((P \rightarrow Q) \rightarrow P) \rightarrow P)} \lambda}{s, p \vdash s(\lambda r.p) : \emptyset} \text{eval}}{\frac{s, p \vdash \text{ex-falso-Q}(s(\lambda r.p)) : Q}{s \vdash \lambda p.\text{ex-falso-Q}(s(\lambda r.p)) : P \rightarrow Q} \lambda} \text{eval}$$

In other words, in context s we can construct a term of type $P \rightarrow Q$. For conciseness let us denote the constructed term as $g : P \rightarrow Q$. We can then conclude as follows:

$$\frac{s, t : ((P \rightarrow Q) \rightarrow P) \vdash t \quad s, t \vdash g : (P \rightarrow Q)}{s, t, \vdash t(f) : P} \text{eval}$$

$$\frac{s \vdash s \quad \frac{s \vdash \lambda t.t(f) : (((P \rightarrow Q) \rightarrow P) \rightarrow P)}{s \vdash s(\lambda t.t(f)) : \emptyset} \lambda}{\vdash \lambda s.s(\lambda t.t(f)) : \neg\neg(((P \rightarrow Q) \rightarrow P) \rightarrow P)} \lambda$$

(iii)

(To make the following slightly more compact we omit the types of some terms after they are first specified)

$$\frac{s, q \vdash \text{inl} : (P \rightarrow Q) \rightarrow ((P \rightarrow Q) + (Q \rightarrow P)) \quad \frac{s : \neg((P \rightarrow Q) + (Q \rightarrow P)), p : P, q : Q \vdash q : Q}{s, q \vdash \lambda p.q : (P \rightarrow Q)} \lambda}{s, q \vdash \text{inl}(\lambda p.q) : ((P \rightarrow Q) + (Q \rightarrow P))} \text{eval}$$

$$\frac{s, q \vdash s \quad \frac{s, q \vdash \text{inl}(\lambda p.q) : ((P \rightarrow Q) + (Q \rightarrow P))}{s, q \vdash s(\text{inl}(\lambda p.q)) : \emptyset} \text{eval}}{\vdash \lambda s.s(\text{inl}(\lambda p.q)) : \neg\neg(((P \rightarrow Q) \rightarrow P) \rightarrow P)} \lambda$$

we can then continue with

$$\frac{s, q \vdash \text{inr} : (Q \rightarrow P) \rightarrow ((P \rightarrow Q) + (Q \rightarrow P)) \quad \frac{s, q \vdash s(\text{inl}(\lambda p.q)) : \emptyset \quad s, q \vdash \text{ex-falso-P} : \emptyset \rightarrow P}{s, q \vdash \text{ex-falso-P}(s(\text{inl}(\lambda p.q))) : P} \text{eval}}{\frac{s \vdash \text{inr} : (Q \rightarrow P) \rightarrow ((P \rightarrow Q) + (Q \rightarrow P)) \quad \frac{s \vdash \lambda q.\text{ex-falso-P}(s(\text{inl}(\lambda p.q))) : Q \rightarrow P}{s \vdash \text{inr}(\lambda q.\text{ex-falso-P}(s(\text{inl}(\lambda p.q)))) : ((P \rightarrow Q) + (Q \rightarrow P))} \lambda}{\vdash \lambda s.\text{inr}(\lambda q.\text{ex-falso-P}(s(\text{inl}(\lambda p.q)))) : \neg\neg(((P \rightarrow Q) \rightarrow P) \rightarrow P)} \text{eval}$$

and finally conclude with

$$\frac{\frac{s \vdash s \quad s \vdash \text{inr}(\lambda q. \text{ex-falso-P}(s(\text{inl}(\lambda p. q)))) : ((P \rightarrow Q) + (Q \rightarrow P))}{s \vdash s(\text{inr}(\lambda q. \text{ex-falso-P}(s(\text{inl}(\lambda p. q)))) : \emptyset} \text{eval}}{\vdash \lambda s. s(\text{inr}(\lambda q. \text{ex-falso-P}(s(\text{inl}(\lambda p. q)))) : \neg \neg ((P \rightarrow Q) + (Q \rightarrow P))} \lambda$$

(iv)

This is following the same kind of idea as the previous exercise

$$\frac{\frac{s \vdash s \quad \frac{s : \neg(P + \neg P), p : P \vdash s(\text{inl}(p)) : \emptyset}{s \vdash \lambda p. s(\text{inl}(p)) : \neg P} \lambda}{s \vdash s(\text{inr}(\lambda p. s(\text{inl}(p)))) : \emptyset} \text{eval}}{\vdash \lambda s. s(\text{inr}(\lambda p. s(\text{inl}(p)))) : \neg \neg (P + \neg P)} \lambda$$

d

(i)

The whole idea is that we will create terms $P \rightarrow (\neg \neg P \rightarrow P)$ and $\neg P \rightarrow (\neg \neg P \rightarrow P)$ then use our induction principle for coproducts to get our final term:

$$\frac{\frac{p : P, q : \neg \neg P \vdash p : P}{\vdash \lambda p \lambda q. p : P \rightarrow (\neg \neg P \rightarrow P)} \lambda^2}{\frac{r : \neg P, q : \neg \neg P \vdash q(r) : \emptyset \quad r : \neg P, q : \neg \neg P \vdash \text{ex-falso-P} : \emptyset \rightarrow P}{r, q \vdash \text{ex-falso-P}(q(r)) : P} \text{eval}}{\vdash \lambda r. \lambda q. \text{ex-falso-P}(q(r)) : \neg P \rightarrow (\neg \neg P \rightarrow P)} \lambda^2$$

to conclude apply the induction principle for coproducts.

(ii)

$$(\neg \neg (Q \rightarrow P) \rightarrow ((P + \neg P) \rightarrow (Q \rightarrow P)) \times (((P + \neg P) \rightarrow (Q \rightarrow P)) \rightarrow \neg \neg (Q \rightarrow P))$$

To begin we want to construct a term of type $(\neg \neg (Q \rightarrow P) \rightarrow ((P + \neg P) \rightarrow (Q \rightarrow P)))$. Our plan is to use our induction principle to construct a term of type $((P + \neg P) \rightarrow (Q \rightarrow P))$ in context $r : \neg \neg (Q \rightarrow P)$. We start by constructing a term of type $P \rightarrow (Q \rightarrow P)$ in this context:

$$\frac{\frac{r : \neg \neg (Q \rightarrow P), p : P, q : Q \vdash p : P}{r, p \vdash \lambda q. p : (Q \rightarrow P)} \lambda}{r \vdash \lambda p. (\lambda q. p) : P \rightarrow (Q \rightarrow P)} \lambda$$

We now want a term of type $\neg P \rightarrow (Q \rightarrow P)$ in this context:

$$\begin{array}{c}
\frac{r, n, q \vdash \text{ex-falso-P} : \emptyset \rightarrow P \quad \frac{r, n, q \vdash r \quad \frac{r : \neg\neg(Q \rightarrow P), n : \neg P, q : Q, \varphi : (Q \rightarrow P) \vdash \varphi(q)(n) : \emptyset}{r, n, q \vdash \lambda\varphi.\varphi(q)(n) : \neg(Q \rightarrow P)} \text{eval}}{r, n, q \vdash r(\lambda\varphi.\varphi(q)(n)) : \emptyset} \text{eval} \\
\frac{\frac{r, n, q \vdash \text{ex-falso-P}(r(\lambda\varphi.\varphi(q)(n))) : P}{r \vdash \lambda n.\lambda q.\text{ex-falso-P}(r(\lambda\varphi.\varphi(q)(n))) : \neg P \rightarrow (Q \rightarrow P)} \lambda^2}{r \vdash \lambda n.\lambda q.\text{ex-falso-P}(r(\lambda\varphi.\varphi(q)(n))) : \neg P \rightarrow (Q \rightarrow P)} \lambda^2
\end{array}$$

Using our induction principle on coproducts we now have a term of the first type in our target product type. We now aim to produce a term of the second type in our target product type to then use the pair map for products to finish things off (in what follows we use $h : P \rightarrow (Q \rightarrow P)$ to denote the term we constructed first, namely $\lambda p.(\lambda q.p)$, producing constant maps):

$$\frac{z : (P + \neg P) \rightarrow (Q \rightarrow P), r : \neg(Q \rightarrow P), p : P \vdash h(p) : (Q \rightarrow P) \quad \frac{z, r, p \vdash r}{z, r, p \vdash r(h(p)) : \emptyset} \text{eval}}{z, r \vdash \lambda p.r(h(p)) : \neg P} \lambda$$

So we have

$$\frac{z, r \vdash \text{inr}(\lambda p.r(h(p))) : P + \neg P \quad \frac{z, r \vdash r}{z, r \vdash z(\text{inr}(\lambda p.r(h(p)))) : Q \rightarrow P} \text{eval}}{z, r \vdash r(z(\text{inr}(\lambda p.r(h(p)))) : \emptyset} \text{eval} \\
\frac{\frac{z, r \vdash r(z(\text{inr}(\lambda p.r(h(p)))) : \emptyset}{\vdash \lambda z.\lambda r.r(z(\text{inr}(\lambda p.r(h(p)))) : (((P + \neg P) \rightarrow (Q \rightarrow P)) \rightarrow \neg\neg(Q \rightarrow P))} \lambda^2}{\vdash \lambda z.\lambda r.r(z(\text{inr}(\lambda p.r(h(p)))) : (((P + \neg P) \rightarrow (Q \rightarrow P)) \rightarrow \neg\neg(Q \rightarrow P))} \lambda^2$$

e

(i)

In what follows we use h to denote the term of type $P \rightarrow \neg\neg P$ we constructed in b(i) above.

$$\frac{q : \neg\neg\neg P, p : P \vdash h(p) : \neg\neg P \quad \frac{q : \neg\neg\neg P, p : P \vdash q : \neg\neg\neg P}{q, p \vdash q(h(p)) : \emptyset} \text{eval}}{\vdash \lambda q.\lambda p.q(h(p)) : \neg\neg\neg P \rightarrow \neg P} \lambda^2$$

(ii)

By b(ii) above we have a term of type $(P \rightarrow \neg\neg Q) \rightarrow (\neg\neg P \rightarrow \neg\neg\neg Q)$. By our previous part we have a term of type $\neg\neg\neg Q \rightarrow \neg\neg Q$ and by b(i) we have a term of type $P \rightarrow \neg\neg P$, which we can respectively post and pre compose our b(ii) term with, on a term of type $P \rightarrow \neg\neg Q$, to get a complete term of type $(P \rightarrow \neg\neg Q) \rightarrow (P \rightarrow \neg\neg Q)$, as desired.

(iii)

We have

$$\frac{s : \neg P, r : (\neg\neg P) \times (\neg\neg Q) \vdash \text{pr}_1(r) : \neg\neg P \quad s : \neg P, r : (\neg\neg P) \times (\neg\neg Q) \vdash s : \neg P}{s, r \vdash \text{pr}_1(r)(s) : \emptyset} \text{eval}$$

$$\frac{}{s \vdash \lambda r. \text{pr}_1(r)(s) : \neg((\neg\neg P) \times (\neg\neg Q))} \lambda$$

Calling the above term $k : \neg((\neg\neg P) \times (\neg\neg Q))$ which we constructed in context s we can construct a term of type $\neg\neg P$:

$$\frac{s : \neg P, q : \neg\neg((\neg\neg P) \times (\neg\neg Q)) \vdash q(k) : \emptyset}{q \vdash \lambda s. q(k) : \neg\neg P} \lambda$$

We can perform the same constructions to see that $t : \neg Q \vdash l : \neg((\neg\neg P) \times (\neg\neg Q))$ which we can use to get $q \vdash \lambda t. q(l) : \neg\neg Q$. We can then use our pairing function on the terms we constructed in context q of types $\neg\neg P$ and $\neg\neg Q$, respectively, to get a term of type $(\neg\neg P) \times (\neg\neg Q)$ in context q . We then perform a λ abstraction to finally get a term of our desired type, namely $\neg\neg((\neg\neg P) \times (\neg\neg Q)) \rightarrow (\neg\neg P) \times (\neg\neg Q)$.

f

(i)

We construct a term of type $\neg\neg(P \times Q) \rightarrow (\neg\neg P) \times (\neg\neg Q)$ by constructing terms of types $\neg\neg(P \times Q) \rightarrow \neg\neg P$ and $\neg\neg(P \times Q) \rightarrow \neg\neg Q$:

$$\frac{z : \neg\neg(P \times Q), r : \neg P, (p, q) : P \times Q \vdash \text{pr}_1(p, q) : P \quad z, r, (p, q) \vdash r}{z, r \vdash z} \frac{}{z, r, (p, q) \vdash r(\text{pr}_1(p, q)) : \emptyset} \text{eval}$$

$$\frac{z, r \vdash z(\lambda(p, q). r(\text{pr}_1(p, q))) : \emptyset}{z \vdash \lambda r. z(\lambda(p, q). r(\text{pr}_1(p, q))) : \neg\neg P} \lambda + \text{eval}$$

The way we produce our term of type $\neg\neg(P \times Q) \rightarrow \neg\neg Q$ is the same as the above, just replacing P with Q and pr_1 with pr_2 . We finish with some λ abstractions and using the induction principle for product types.

We now set out to construct a term of type $(\neg\neg P) \times (\neg\neg Q) \rightarrow \neg\neg(P \times Q)$:

$$\frac{y : (\neg\neg P) \times (\neg\neg Q), x : \neg(P \times Q), p : P, q : Q \vdash x(p \times q) : \emptyset}{y, x, q \vdash \text{pr}_1(y) : \neg\neg P} \lambda$$

$$\frac{y, x, q \vdash \text{pr}_1(y)(\lambda p. (x(p \times q))) : \emptyset}{y, x, q \vdash \text{pr}_1(y)(\lambda p. (x(p \times q))) : \neg P} \text{eval}$$

$$\frac{}{y, x \vdash \lambda q. \text{pr}_1(y)(\lambda p. (x(p \times q))) : \neg Q} \lambda$$

One we are in the above situation we can evaluate our term using $y, x \vdash \text{pr}_2(y) : \neg\neg Q$ to derive a term of the empty type in context y, x . We then λ -abstract twice to get our desired term.

(ii)

(iii)